Predicting Irrigation Need

Master's Degree in Artificial Intelligence
Machine Learning
University Of Verona - A.Y. 2025/2026

Carlo Stasi VR543606

June 2026

Contents

1 Introduction

An irrigation manager cannot inspect every field by hand. What they have instead is data: soil moisture and pH from probes in the ground, temperature, humidity and sunlight from a weather station, and what is already known about each plot, namely the crop growing on it, its stage, the system that waters it. This project asks whether those readings are enough to decide, plot by plot, how much water a field needs.

We treat it as a three-class problem, $Y \in \{\text{Low}, \text{Medium}, \text{High}\}$, over 630,000 entries described by 19 attributes. Two properties of the data shape every decision that follows. The critical `High` class is 3.3% of the total, and the two kinds of error do not cost the same: watering a field that did not need it wastes water, while failing to water one that did loses the crop.

The following is a benchmark of five models under two training regimes, tested on a single common test set and read against two purposefully non-ambitious baselines. The most important result is the explanation of why that figure is reachable at all, which turns out to say more about the data than about the models.

2 Motivation and Rationale

2.1 Context and Agronomic Problem

Soil and micro-climate vary from one plot to the next, and across the season. A schedule that waters every field on the same day therefore drowns some and starves others [1]. The alternative is to decide, per plot, which means turning a row of sensor readings into an irrigation level. This mapping is what a classifier must learn.

2.2 The Imbalance Challenge and Engineering Significance

Two things make the problem harder than it might seem at first glance:

- **Asymmetric risk and class skew:** The `High` class accounts for $\approx 3.3\%$ of the population ($n_{\text{High}} =$ 21,009 of 630,000), and a symmetric objective treats a class that rare as noise [2]. But, as mentioned, the two errors are not worth the same. Over-irrigating costs water while failing to detect a drought costs the crop. Minority-class **recall**, read next to the **macro F1-score**, is therefore the operational criterion of this project.
- **The cost of the non-linear estimators:** The two geometric models are expensive in different ways, and only one of them is actually blocked. Training an SVM with a Radial Basis Function kernel costs between $\mathcal{O}(N^2)$ and $\mathcal{O}(N^3)$ in the number of rows, which puts the 504,000-row training almost out of the picture right away; that limit is real and it stands. K-NN is different, as it has no training cost and instead spends everything at prediction, at $\mathcal{O}(N_{\text{train}} \cdot N_{\text{test}} \cdot d)$. This work treated that as a second wall for a long time, until it was measured: one pass over the test set takes 31 seconds and a full grid search over K takes fifteen minutes. The constraint on K-NN is therefore not feasibility but price: it answers a query in 30.6 seconds where XGBoost needs 0.1 and a single tree too little to measure, and it pays that cost on every query, for as long as the model is deployed.

3 State of The Art (SOTA)

3.1 Current Methodologies

Two families dominate the machine learning literature on smart farming [1]:

1. **Distance and kernel-based classifiers.** K-NN [3] and RBF-kernel SVMs [4] are non-parametric, which suits the smooth, continuous relationships expected of telemetry.
2. **Tree-based ensembles.** Random Forest [5] and gradient-boosted trees, XGBoost [6] in particular, are the state of the art on tabular data. They handle mixed feature types without preprocessing and split on thresholds, which is exactly the shape an agronomic rule tends to take.

3.2 Weaknesses of SOTA Methods

Each family carries a weakness that this dataset highlights:

- **Cost at scale.** The RBF SVM's training time grows as $N^{2.09}$ on this data, measured from 32,000 to 96,000 rows: one fit on the full partition costs about 1.6 hours and the eighteen of its grid search

over twelve, so what rules it out there is the tuning and not the fit. K-NN remains feasible but expensive, at the prices measured in Section 2.

- **Geometric rigidity.** A linear SVM scales comfortably but cannot isolate a threshold on temperature or moisture, and its recall on the minority class suffers for it.
- **Uniform loss bias.** Symmetric losses gain global accuracy at the cost of the rare class [2]. Synthetic over-sampling such as SMOTE [7] is the standard remedy, but generating minority instances is expensive, which is why this project instead employs undersampling and cost-sensitive weights.

3.3 Proposed Methodological Enhancements

Three interventions follow from those weaknesses.

- **Physics-informed feature engineering.** Three non-linear interaction terms drawn from soil thermodynamics and water balance are added to the design matrix, so that a low-capacity linear model can reach a boundary it could not otherwise represent. We state this as a hypothesis to be tested, not as a gain already banked: Section 6.9 removes each term and refits, and finds that only one of the three survives.
- **Dual-arena benchmarking.** The data is split once, on its natural class distribution, into a training partition ($N = 504{,}000$) and a common test set ($N = 126{,}000$). Two arenas then differ only in the training regime: a class-balanced subsample of the training partition ($N = 50{,}421$), and the full partition under cost-sensitive weighting. Every model is fitted in both arenas except the RBF SVM, whose tuning cost rules out the second (Section 3). Because the two are scored on the same held-out rows, the difference between a model's two entries measures the training regime and nothing else. Section 5.2 gives the construction.
- **Separating the tuning objective from the operational one.** Every hyperparameter is chosen by grid search under stratified cross-validation against the **macro F1-score**, while minority-class **recall** is reported separately as the criterion the deployment decision turns on. The two are kept apart deliberately. A search needs one scalar; that scalar is not what a grower cares about, and Section 6 shows the two disagreeing on the ranking.

4 Objectives

The project sets itself four goals.

- **A pipeline that faces the imbalance directly.** Build and evaluate a multiclass pipeline for irrigation demand in which the minority class and the cost of running the models are treated as design constraints rather than things to solve later on.
- **Features justified by measurement, not by agronomy alone.** Derive interaction terms from physical reasoning, then test whether they earn their place. The aggregate *Total Water Input* was designed on the same reasoning and discarded once measured, being almost collinear with `Rainfall_mm` ($r = 0.998$, Section 5.3).
- **A benchmark with a floor under it.** Compare five models (K-NN, linear SVM, RBF SVM, Random Forest, XGBoost) across the two trainings, and account for why each behaves as it does. Every score is read against two un-tuned baselines, a majority-class classifier and a single decision tree, so that each value is compared to the baseline it has to surpass.
- **Diagnostics instead of only scores.** Evaluate with metrics that survive class imbalance, namely macro F1 and recall, on the minority class, never accuracy alone, and read the results through a correlation matrix, confusion matrices, one-vs-rest ROC curves and feature importances.

5 Methodology

5.1 Dataset Description and Class Imbalance

The dataset consists of $N = 630{,}000$ entries described by 19 attributes plus an integer identifier and the target variable. No missing values and no duplicate records were found. The predictive attributes split into 11 continuous variables and 8 categorical variables.

The experimental data originates from the **Kaggle Playground Series** (Season 6, Episode 4) [8], synthetically generated by a deep generative pipeline trained on the real-world *Irrigation Water Requirement*

Prediction dataset. The dataset's goal is to forecast plot-level irrigation demand, where each individual record represents a single agricultural plot under specific environmental, soil, and management conditions at an arbitrary time step.

Table 1: Descriptive statistics of the 11 continuous attributes.

Attribute	**Mean**	**Std**	**Min**	**Median**	**Max**	**Skew**
Soil_pH	6.48	0.92	4.80	6.44	8.20	0.07
Soil_Moisture (%)	37.30	16.38	8.00	37.75	64.99	−0.06
Organic_Carbon (%)	0.92	0.37	0.30	0.91	1.60	0.11
Electrical_Conductivity	1.75	0.95	0.10	1.74	3.50	0.05
Temperature_C	27.00	8.62	12.00	26.96	42.00	−0.00
Humidity (%)	61.56	19.71	25.00	61.65	94.99	−0.09
Rainfall_mm	1462.21	612.99	0.38	1467.16	2499.69	−0.12
Sunlight_Hours	7.51	2.00	4.00	7.58	11.00	−0.04
Wind_Speed_kmh	10.38	5.69	0.50	10.48	20.00	−0.03
Field_Area_hectare	7.52	4.22	0.30	7.38	15.00	0.05
Previous_Irrigation_mm	62.32	34.25	0.02	61.15	119.99	−0.02

Every continuous attribute in Table 1 is close to symmetric (skewness near zero, mean and median all but coincident), and there are no extreme outliers, both consistent with a synthetic generator. The measurement scales, by contrast, span three orders of magnitude. This is what makes the z-score standardization of Section 5.2 necessary for the distance- and margin-based estimators, and the gap between `Rainfall_mm` and `Previous_Irrigation_mm` ($\mu = 62.32$) is what motivates the multicollinearity analysis of Section 5.3.

Table 2: Categorical attributes: cardinality and category frequencies.

Attribute	**Categories**	**Levels**
Soil_Type	Sandy, Clay, Loamy, Silt	4
Crop_Type	Sugarcane, Rice, Cotton, Maize, Wheat, Potato	6
Crop_Growth_Stage	Harvest, Flowering, Vegetative, Sowing	4
Season	Kharif, Rabi, Zaid	3
Irrigation_Type	Canal, Sprinkler, Rainfed, Drip	4
Water_Source	Reservoir, River, Groundwater, Rainwater	4
Mulching_Used	No, Yes	2
Region	South, West, East, Central, North	5

As summarized in Table 2, the categorical features show near-uniform marginal distributions across all distinct levels, avoiding the need for grouping or pruning. The low categorical cardinality results in a controlled one-hot encoding expansion of the feature space. Interestingly, despite their uniform distributions, operational features such as `Mulching_Used` and `Crop_Growth_Stage` emerge as primary decision drivers in the tree-based feature importance of Figure 9.

Table 3: Target variable distribution (`Irrigation_Need`).

Class	**Instances**	**Share**
Low (0)	369,917	58.72 %
Medium (1)	239,074	37.95 %
High (2)	21,009	3.33 %
Total	630,000	100.00 %

The target distribution in Table 3 shows a rather severe class imbalance: the critical `High` class accounts for only 3.33% of the population (n = 21,009), producing a `Low`-to-`High` ratio of 17.6 : 1: the consequence is that a "trivial" classifier predicting the majority class for every instance would already reach 58.72% accuracy while detecting no drought emergency at all. Accuracy is therefore useless on this task,

which motivates our choice of Macro F1-Score and minority-class Recall as the optimization targets in Sections 5.4 and 6, both of which penalize the asymmetric cost of undetected water stress.

5.2 Data Pre-processing and Transformation Pipeline

A scikit-learn [9] `ColumnTransformer` pipeline was constructed to ensure zero data leakage across splits:

1. **Categorical Encoding:** The 8 qualitative features were transformed using One-Hot Encoding (`drop='first'`, `sparse_output=False`), expanding the categorical space.
2. **Continuous Feature Standardization:** All 14 continuous features (11 original attributes plus the 3 engineered terms) were standardized via z-score normalization:

$$z = \frac{x - \mu}{\sigma} \tag{1}$$

   where μ and σ are estimated strictly on the training partition.
3. **Stratified Splitting:** Train and test partitions were generated using an 80/20 stratified split (`random_state=42`) to preserve target class proportions, producing 504,000 training and 126,000 test instances.
4. **Balancing Confined to the Training Side:** The split is performed *before* any class balancing, and the 1:1:1 subsampling of the balanced lot is applied to the training partition only. Balancing the dataset as a whole would balance the test set with it, and every reported score would then be measured on an artificial population that never occurs in the field.

The balancing is a *random undersampling* of the two majority classes. The per-class quota is set to the size of the rarest class available in the training partition, so the balanced lot keeps every one of the 16,807 `High` rows that survive the split and draws 16,807 `Low` and `Medium` rows without replacement, producing the exact 1:1:1 composition of Table 4. The per-class cap of 21,000 is therefore not binding: the entire minority class is smaller than it and no instance is duplicated and none is generated, which keeps the design consistent with the position taken in Section 3, where SMOTE [7] was set aside on cost grounds; the same end is reached by discarding majority rows instead of synthesizing minority ones.

Table 4: Composition of the two training arenas and of the common test set. The balanced arena keeps the whole of the minority class and undersamples the two majority classes down to it; the test set is the same in both arenas and retains the natural distribution of Table 3.

Partition	**Low**	**Medium**	**High**	**Total**
Training — full arena	295,934	191,259	16,807	504,000
Training — balanced arena	16,807	16,807	16,807	50,421
fraction retained	5.68 %	8.79 %	100.00 %	10.00 %
Common test set	73,983	47,815	4,202	126,000
class share	58.72 %	37.95 %	3.33 %	100.00 %

The cost of the balanced arena is thus paid in data instead of in fabricated points: 453,579 training instances, 90% of the partition, are discarded. That is affordable only because discarding them is what makes the RBF SVM tunable at all. The two arenas therefore cover the two standard treatments of class imbalance (resampling and cost-sensitive weighting) on a common test set, rather than committing the study to either one.

5.3 Domain-Specific Feature Engineering and Multicollinearity Mitigation

To introduce physical domain knowledge into the input space, three non-linear interaction terms were engineered. A fourth quantity, the atmospheric evaporation proxy

$$E = \frac{\text{Temperature\_C}}{\text{Humidity} + 10^{-5}}, \tag{2}$$

is used only as an intermediate term: it measures thermal moisture extraction and enters the design matrix through FE_3 below, but is not kept as a column of its own, so that atmospheric demand is represented once, in the ratio form that is agronomically meaningful. Recovering E as a standalone feature is a natural ablation, and Section 6.9 carries it out. The three retained terms are:

- **Thermal Impact Index (FE_1):** Models solar radiative exposure:

$$FE_1 = \text{Temperature\_C} \times \text{Sunlight\_Hours} \tag{3}$$

- **Biochemical Soil Health Index (FE_2):** Relates organic carbon to absolute deviation from optimal agronomic pH (6.5), compressed via log-transform:

$$FE_2 = \ln\left(1 + \frac{\text{Organic\_Carbon}}{|\text{Soil\_pH} - 6.5| + 10^{-5}}\right) \tag{4}$$

- **Water Deficit Ratio (FE_3):** Captures atmospheric demand against soil moisture:

$$FE_3 = \frac{E}{\text{Soil\_Moisture} + 10^{-5}} \tag{5}$$

A fourth candidate, an aggregate *Total Water Input* defined as Rainfall_mm + Previous_Irrigation_mm, was designed and then discarded after empirical inspection. The two components of the sum differ in dispersion by a factor of 17.9 ($\sigma = 612.99$ against $\sigma = 34.25$), so the sum is almost entirely driven by its larger constituent: it correlates at $r = 0.998$ with `Rainfall_mm` and at only $r = 0.069$ with `Previous_Irrigation_mm`. The aggregate is therefore a near-duplicate of a column already present in the matrix, adding one dimension and a strong collinearity while contributing essentially no information that `Rainfall_mm` does not already carry. Since the two original attributes are mutually uncorrelated (Figure 1, $r = 0.013$) and individually informative, the aggregate alone was dropped and both constituents were kept. This empirically confirms the original concern of Table 1 regarding the magnitude gap between the two attributes.

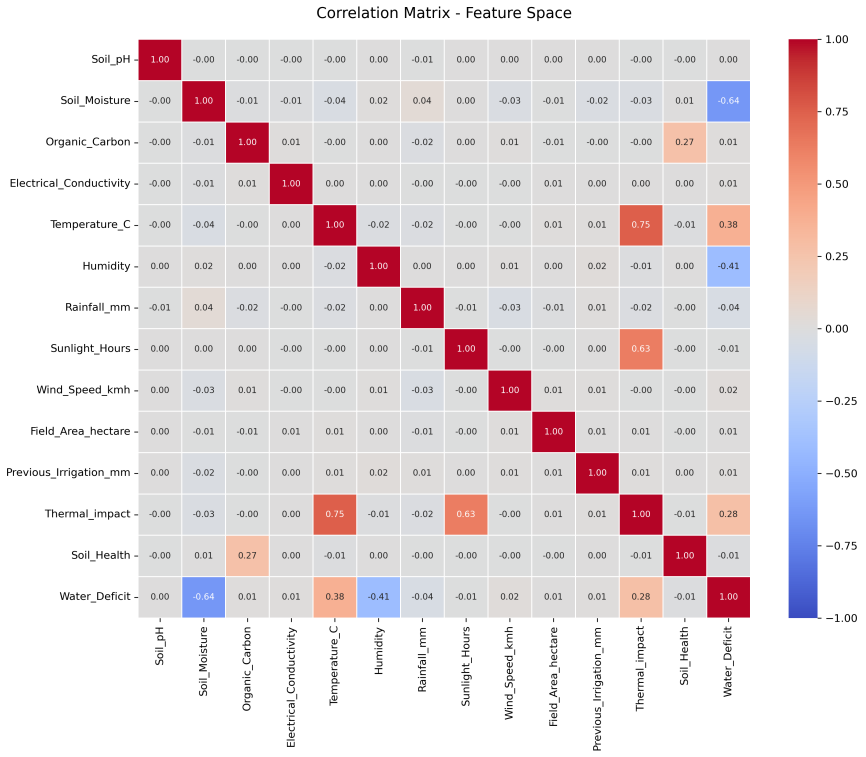

Figure 1: Correlation matrix of the 14 continuous attributes, computed on the 504,000-row training partition. The 11 original attributes are uncorrelated to within $|r| \leq 0.04$, `Rainfall_mm` against `Previous_Irrigation_mm` among them ($r = 0.01$), which is what left both of them in the design matrix. Every off-diagonal cell of any magnitude involves an engineered term and its own constituents: FE_1 against `Temperature_C` (0.75) and `Sunlight_Hours` (0.63), FE_3 against `Soil_Moisture` (-0.64) and `Humidity` (-0.41). The independence of the raw attributes is itself a property of the synthetic generator, and Section 6.6 shows what it implies.

5.4 Model Architectures and Hyperparameter Tuning

All five architectures operate on the same 38-dimensional encoded input (14 standardized continuous attributes plus 24 one-hot indicators) and were optimized using 3-fold stratified `GridSearchCV` maximizing the Macro F1-Score. Table 5 reports the complete search space together with the configuration selected in each experimental lot.

Table 5: Search space and selected hyperparameters per lot. Bold marks a value selected at an extreme of a grid of three or more levels. A dash marks a model not trained in that arena. The two baselines of Section 6 carry no tuned hyperparameters and are not included: they are left un-tuned by design, so that they measure a floor rather than compete.

Model	Hyperparameter	Search grid	Balanced	Full data
K-NN	`n_neighbors`	10 values in $[3, 200]$	31	11
Linear SVM	`C`	0.1, 1.0, 10.0	**10.0**	**0.1**
RBF SVM	`C`	0.1, 1.0, 10.0	**10.0**	—
	`gamma`	scale, auto	auto	—
Random Forest	`n_estimators`	100, 200, 300	**100**	**300**
	`max_depth`	10, 20, 30, None	**None**	20
	`min_samples_split`	2, 5, 10	5	5
	`min_samples_leaf`	1, 2, 4	**1**	**1**
XGBoost	`n_estimators`	100, 150, 200	**200**	**200**
	`max_depth`	4, 6	6	4
	`learning_rate`	0.05, 0.1	0.1	0.1
	`subsample`	0.8, 1.0	1.0	0.8

Cost sensitivity is injected differently across families: linear SVM and Random Forest use class_weight='balanced', i.e. inverse-frequency weights $w_c = \frac{N}{3 \cdot n_c}$; XGBoost receives the equivalent per-sample weights through `compute_sample_weight`. K-NN and RBF SVM are instead left unweighted, since in the balanced arena the 1:1:1 subsampling already removes the skew from the data they are fitted on. Both are still scored on the natural distribution of the common test set, so any mismatch between the prior they implicitly learn and the prior they meet at test time surfaces directly in Section 6.

Three observations follow from Table 5:

First, the optimal regularization parameter is driven primarily by the training setup rather than the choice of kernel. When tuned over identical grids across both regimes, the Linear SVM mirrors this behavior directly: on the balanced subsample (N = 50,421), it selects the maximum available penalty ($C = 10.0$), matching the choice of the RBF SVM. Under full-partition training with inverse-frequency class weighting ($N \approx 504{,}000$), the same estimator selects the minimum penalty ($C = 0.1$). In the smaller, balanced setting, class overlap is reduced, permitting a narrower margin that fits the local boundary tightly. Conversely, on the weighted full dataset, a linear separator cannot resolve the non-linear decision surface (detailed in Section 6); tightening the margin allows heavily up-weighted minority instances to distort the hyperplane without achieving better class separation. Consequently, stronger regularization ($C = 0.1$) penalizes margin violations without yielding better class separation, reflecting a boundary solution on the grid rather than an unconstrained optimum.

Second, the two ensemble methods adapt to the tenfold expansion in training volume in fundamentally divergent ways. Random Forest expands its global capacity while restricting individual tree depth, moving from 100 unconstrained trees (empirically varying between 22 and 36 levels on the balanced subset) to 300 trees constrained at the maximum evaluated depth of 20. Because all 300 trees hit this bound, deeper configurations were actively penalized during cross-validation, indicating that the estimator favored horizontal capacity (ensemble size) over vertical tree depth. In contrast, XGBoost behaves conservatively by regularizing further: maximum depth decreases from 6 to 4 and the row subsampling ratio drops from 1.0 to 0.8 (with boosting rounds fixed at 200 and learning rate $\eta = 0.1$). With an order of magnitude more observations, gradient boosting favors shallower, randomized base learners—a behavior consistent with refining the estimate of a fixed-complexity decision boundary while controlling additive model variance. Because leaf constraints remained identical across regimes (`min_samples_split` = 5, `min_samples_leaf` = 1), capacity adaptation was governed entirely by tree depth and ensemble size.

Third, this divergence produces an immense disparity in model footprint. On the full partition, the

Random Forest expands to 2,784,223 leaves across 300 trees (a 494 MB footprint)—a 480× storage penalty over XGBoost (1.03 MB) that, as shown in Section 6, purchases no measurable gain in generalization. As demonstrated in Section 6, this substantial computational and memory overhead yields negligible generalization benefit.

6 Experiments & Results

6.1 Evaluation Protocol

All empirical results presented in this section are evaluated on the unified, holdout test partition ($N = 126{,}000$) defined in Table 4. This set preserves the natural empirical class distribution, remains identical across both experimental regimes, and is strictly disjoint from all model training and tuning phases. Consequently, performance metrics across both regimes are directly comparable, isolating the training configuration as the sole independent variable.

A critical implication of evaluating under the natural class distribution is class scarcity: the minority class (`High`) comprises only 4,202 test instances (3.33%). Because of this limited support, class-conditional precision for `High` exhibits the highest empirical variance among the reported metrics and warrants conservative interpretation.

6.2 Baselines

Given severe class imbalance, overall accuracy provides an uninformative performance lower bound, and high macro-averaged F1 scores require context to evaluate genuine discriminatory power. To establish rigorous empirical baselines, two un-tuned reference models are evaluated alongside the primary architectures:

- **Majority-Class Classifier:** Trivial baseline predicting the dominant class (`Low`) for all instances. While achieving an accuracy of 58.72%, it fails to identify any true positive instances for severe events (zero recall on `High`), yielding a macro F1 baseline of 0.2466.
- **Constrained Decision Tree:** A single, interpretable decision tree restricted to a maximum depth of 8, serving as a transparent non-linear benchmark whose competitive performance is further analyzed in Section 6.6.

6.3 Model Comparison

Tables 6 and 7 summarize the comparative performance across both training regimes. In addition to global macro-averaged metrics, each table explicitly details precision, recall, and F1 score for the minority class (`High`). As discussed in Section 2, aggregated macro averages obscure class-specific performance failures that directly dictate operational viability.

Table 6: Balanced arena: 50,421 training rows at a 1:1:1 class ratio. The Linear SVM is tuned here as well as on the full partition, so the two SVM kernels and the two training regimes form a complete 2×2. *Bal. acc.* is balanced accuracy, the macro average of per-class recall: the metric the competition that published this data scores submissions on, and one that does not always agree with macro F1 (Section 6.4).

Model	Macro F1	Bal. acc.	Accuracy	Prec. High	Rec. High	F1 High
Majority class	0.2466	0.3333	0.5872	0.000	0.000	0.000
Decision tree ($d = 8$)	0.9387	0.9607	0.9771	0.793	0.933	0.857
K-NN	0.6452	0.8058	0.7668	0.230	0.934	0.368
RBF SVM	0.8403	0.9156	0.9122	0.545	0.937	0.689
Linear SVM	0.6833	0.8236	0.8089	0.278	0.924	0.427
Random Forest	0.9433	0.9615	0.9787	0.814	0.931	0.869
XGBoost	0.9484	**0.9674**	0.9797	0.825	0.948	0.882

Table 7: Full-data arena: the entire 504,000-row training partition, with cost-sensitive weighting. Bold marks the leader on each of the two aggregate metrics, and they are not the same models.

Model	Macro F1	Bal. acc.	Accuracy	Prec. High	Rec. High	F1 High
Majority class	0.2466	0.3333	0.5872	0.000	0.000	0.000
Decision tree ($d = 8$)	**0.9682**	0.9595	0.9850	0.961	0.907	0.934
K-NN	0.7514	0.7071	0.8671	0.854	0.376	0.522
Linear SVM	0.7536	0.7156	0.8754	0.797	0.370	0.506
Random Forest	**0.9682**	0.9601	0.9850	0.959	0.909	0.933
XGBoost	0.9560	**0.9671**	0.9817	0.866	0.941	0.902

The balanced arena reproduces exactly the failure the protocol was designed to expose: K-NN and RBF SVM both hold recall on `High` above 0.93, yet their precision collapses to 0.230 and 0.545 respectively: trained on a 1:1:1 subsample and deployed against a 3.3% prior, they over-predict the rare class massively. For every genuine drought the K-NN flags, it raises roughly three false alarms, as Figure 2 counts out. This is not a defect of the estimators but the arithmetic of prior shift, and it is visible only because the test set was left at its natural distribution.

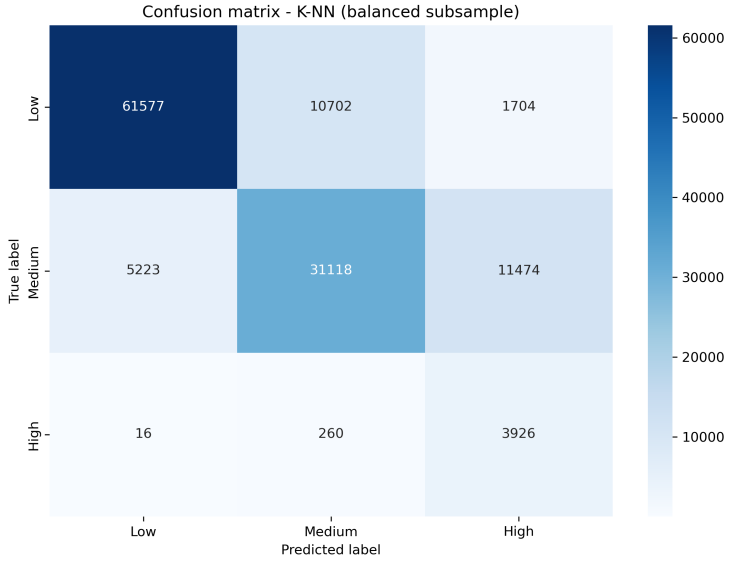

Figure 2: K-NN trained on the balanced arena, scored on the common test set. The `High` column totals 17,104 predictions against 4,202 genuine droughts: the model recovers 3,926 of them, a recall of 0.934, at the price of 13,178 false alarms, 11,474 of which are `Medium` fields. This is the shape of prior shift, not of a badly fitted model.

The mirror image of that failure is in the other arena. Fitted to the full partition without class weights, K-NN reverses completely: precision on `High` climbs from 0.230 to 0.854 while recall falls from 0.934 to 0.376. The Linear SVM does the same thing, from 0.278 and 0.924 to 0.797 and 0.370... Two estimators from unrelated families, one built on distances and one on margins, move by comparable amounts in the same direction under the same change of training distribution, each gaining between 0.07 and 0.09 of macro F1 while losing around 0.6 of recall on the class the project cares about. It is fair to assume that the effect belongs to the distribution the model is fitted on, not to any one model family.

The tuning backs that reading up from an unexpected direction, as K was selected independently in each arena, and the two searches disagree: 31 neighbours on the balanced subsample, 11 on the full partition (Table 5). That gap is the prior shift arriving as a hyperparameter. A large K makes every vote regress towards the global class distribution, and a class holding 3.3% of the population cannot win a plurality that way; a small K keeps the neighbourhood local enough for it to win. Where the training prior is already 1:1:1 the rare class is not fighting the vote, so the search can afford the smoothing, and the lower variance, that a larger K buys. This tuning disparity emerged naturally: stratified cross-validation on macro F1 penalized the high variance of large neighborhoods under severe class skew without explicit prior adjustments.

The ensembles do not move like that: Random Forest gains precision on `High` from 0.814 to 0.959 while its recall barely shifts, 0.931 to 0.909: it turns the extra data into fewer false alarms without giving up droughts. Capacity is what separates the two behaviours... A model able to represent the boundary absorbs the change of prior and spends the additional rows on sharpening it; one that cannot is pushed bodily along the trade-off instead.

Both tables give point estimates, a paired bootstrap over 10,000 resamples of the test set, scoring every model on the same resampled rows so that the variation they share cancels in the difference, places all but one of these gaps outside its own 95% interval: the ordering above is a property of the models and not of which 126,000 rows happened to be held out. The exception is the one this section has been building towards, and Section 6.8 takes it up.

6.4 Macro F1 and the Operational Metric Disagree

The Linear SVM of the full-data arena is the clearest case of a model that ranks well on the aggregate and fails on the criterion this project declared since its macro F1 of 0.7536 places it above the balanced-arena K-NN (0.6452), but that ranking is an artifact of aggregation: its recall on `High` is 0.370, meaning it misses roughly 62.95% of all droughts in the test set, against the 0.93 that same balanced arena delivers. Figure 3 shows the mechanism: the misclassified `High` instances are absorbed almost entirely into `Medium`, a neighbouring class that triggers no emergency response. The K-NN fitted to the same partition scores 0.7514 with a recall of 0.376, within a whisker of the linear model on both counts. The failure belongs to the configuration, not to the estimator.

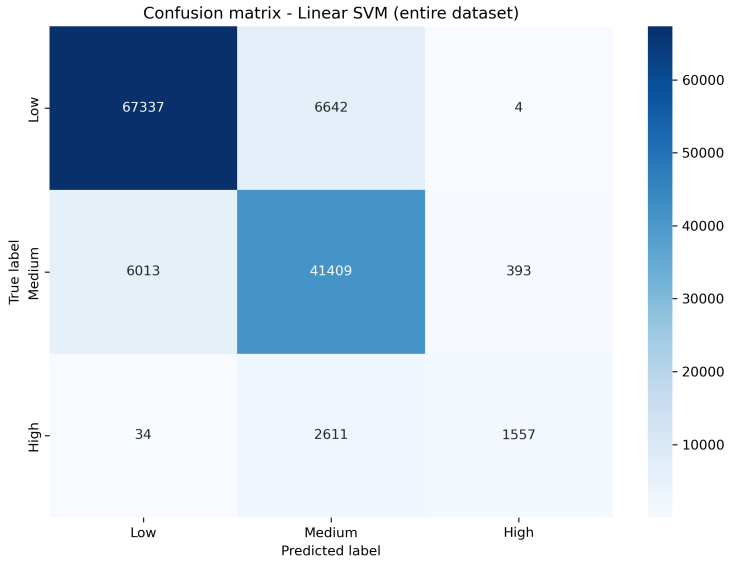

Figure 3: Linear SVM on the full training partition. Of the 4,202 droughts in the test set it recovers 1,557; almost all of the rest are labelled `Medium`. A macro average of 0.7536 does not show this, which is the argument of this subsection.

Under the asymmetric cost assumed in Section 2, where a missed drought is ruinous and over-irrigation is cheap, that configuration is the one to discard despite the aggregate score that favors it. This is the clearest practical argument the benchmark produces: **a single averaged figure ranked the candidates in an order that the operational requirement reverses**, which is why the per-class columns are reported and not just summarized.

The verdict is against the configuration and not against the linear kernel, and Table 6 confirms it... The same estimator, tuned over the same grid under the same inverse-frequency weighting, was fitted to the balanced subsample as well: there it recovers 0.924 of the droughts against 0.370 on the full partition, at a precision of 0.278. A linear boundary is not what blinds the model to droughts; the training distribution is. The two fits also disagree about regularization, in the direction that explains the rest: the balanced arm selects the largest penalty in the grid ($C = 10.0$) and the full-data arm the smallest ($C = 0.1$), so the same linear model, asked to separate 504,000 weighted rows it cannot separate, retreats to the flattest hyperplane the grid allows it.

The sharpest version of this argument was not written by us. The competition that published this data scores submissions on **balanced accuracy**, the macro average of per-class recall, and on that metric the ranking of the full-data models reverses: XGBoost leads at 0.9671 against 0.9601 for the forest and 0.9595 for the tree, where macro F1 puts it last of the three at 0.9560 against their 0.9682. The mechanism is the weighting. XGBoost is fitted with inverse-frequency sample weights, so it buys recall on `High` (0.941 against 0.907 and 0.909) at the price of precision (0.866 against 0.961 and 0.959). Macro F1 charges it for that precision. Balanced accuracy, being an average of recalls, never looks at precision at all.

Two aggregate metrics, one chosen here and one chosen by somebody who had never seen this work, order the same three models in opposite directions. And the external one sits closer to the criterion this project declared on its first page, since it rewards catching droughts and is indifferent to false alarms, which is exactly the asymmetry Section 2 assumes. The tuning objective of this report was the more conservative of the two.

The disagreement is between the models' *decisions*, not between their scores, as Figure 4 draws the one-vs-rest ROC curves of the full-data XGBoost: the areas under them are 0.9973 for `Low`, 0.9950 for `Medium` and 0.9981 for `High`. The ranking of the rare class is very nearly perfect, which is worth holding against the recall of 0.941 the same model achieves on that class, and there's no contradiction between the two: AUC is threshold-free, whereas every score in this section comes from the default arg-max rule. What the pair says is that the information needed to catch almost every drought is already present in the model's scores, and that what limits recall is the decision rule laid over them. Choosing that rule by expected cost instead of by arg-max is the natural continuation, and Section 6.5 carries it out.

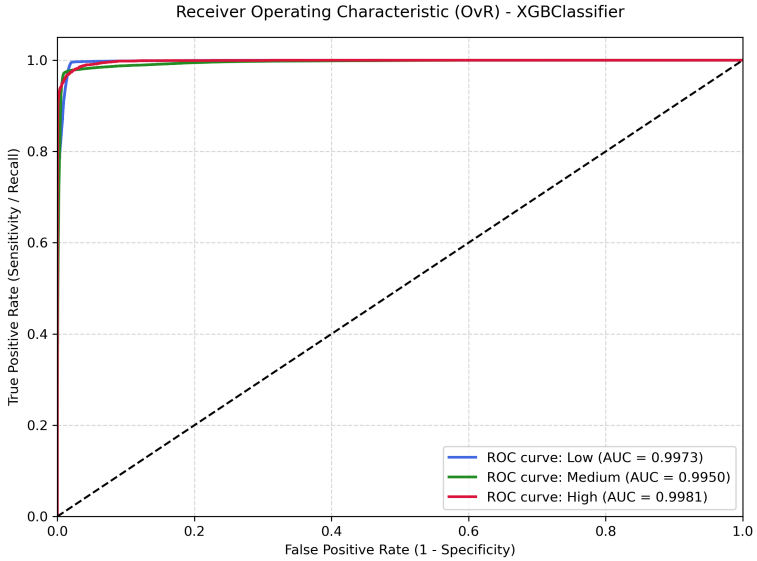

Figure 4: One-vs-rest ROC curves for XGBoost on the full training partition, scored on the common test set. The curve for the rare `High` class is the highest of the three (AUC = 0.9981): the model orders the test instances almost perfectly, and the recall it actually delivers depends on where the decision threshold is placed along that curve.

6.5 Choosing the Decision Rule by Cost

Every score reported so far comes from arg-max over the predicted probabilities. That rule minimizes the expected *number* of errors, which is the right objective only when all errors cost the same, and Section 2 has assumed from its first page that they do not. This subsection asks what the same model does once the rule is matched to the cost.

The rule is a threshold. Predict `High` whenever $P(\texttt{High}) \geq t$ and otherwise take the better of `Low` and `Medium` exactly as before, so t moves nothing except the boundary between catching a drought and missing one. Write λ for the price of a missed drought in needless irrigations: $\lambda = 10$ says that failing to water one field that needed it costs as much as watering ten that did not. The expected cost is then λ times the droughts missed plus the false alarms raised, and t is chosen to minimize it.

Where that minimization happens matters as much as the rule, for the reason Section 6.6 gives about

tree depth. The threshold is chosen on a validation slice held out of the *training* partition, scored by a twin XGBoost carrying the tuned hyperparameters and fitted to the other 80%; only the selected value reaches the test set.

Table 8: Full-data XGBoost under a cost-sensitive threshold. t is chosen on training-side validation for each cost ratio λ; every column to its right is measured on the common test set. The first row is the arg-max rule used everywhere else in this section. *Fields flagged* counts the test instances predicted `High`, out of 4,202 genuine droughts in 126,000 fields.

Rule	t	**Prec. High**	**Rec. High**	**F1 High**	**Macro F1**	**Fields flagged**
arg-max	—	0.866	0.941	0.902	0.9560	4,568
$\lambda = 1$	0.90	0.969	0.902	0.934	**0.9679**	3,912
$\lambda = 2$	0.76	0.946	0.920	0.933	0.9673	4,083
$\lambda = 5$	0.60	0.906	0.934	0.920	0.9626	4,332
$\lambda = 10$	0.38	0.774	0.956	0.855	0.9386	5,193
$\lambda = 20$	0.35	0.739	0.961	0.836	0.9312	5,463
$\lambda = 50$	0.23	0.562	**0.981**	0.714	0.8843	7,339

The first row of Table 8 is surprising. At $\lambda = 1$, where a missed drought and a needless irrigation are priced alike, the cost-optimal threshold is not the 0.5-like boundary arg-max applies but 0.90, and macro F1 *rises* from 0.9560 to 0.9679. The reason is in the training regime: XGBoost is fitted with inverse-frequency sample weights, so its probabilities are already tilted towards the rare class and arg-max over-flags. Correcting the threshold recovers almost the entire gap that separated XGBoost from the depth-8 tree in Section 6.8 (-0.0123), which says that most of that gap was never a deficit of the model at all. It was the decision rule laid on top of it.

Past $\lambda = 2$ the threshold falls and the trade begins in earnest. Recall on `High` climbs from 0.902 to 0.981 while precision drops from 0.969 to 0.562, and the number of fields flagged rises from 3,912 to 7,339. In the units the grower cares about: 331 additional droughts are caught at the price of 3,427 additional fields watered, an average of roughly ten needless irrigations per drought recovered. That is the exchange rate this project has been assuming qualitatively since Section 2, now with a number on it, and Figure 5 shows where each ratio lands on the frontier.

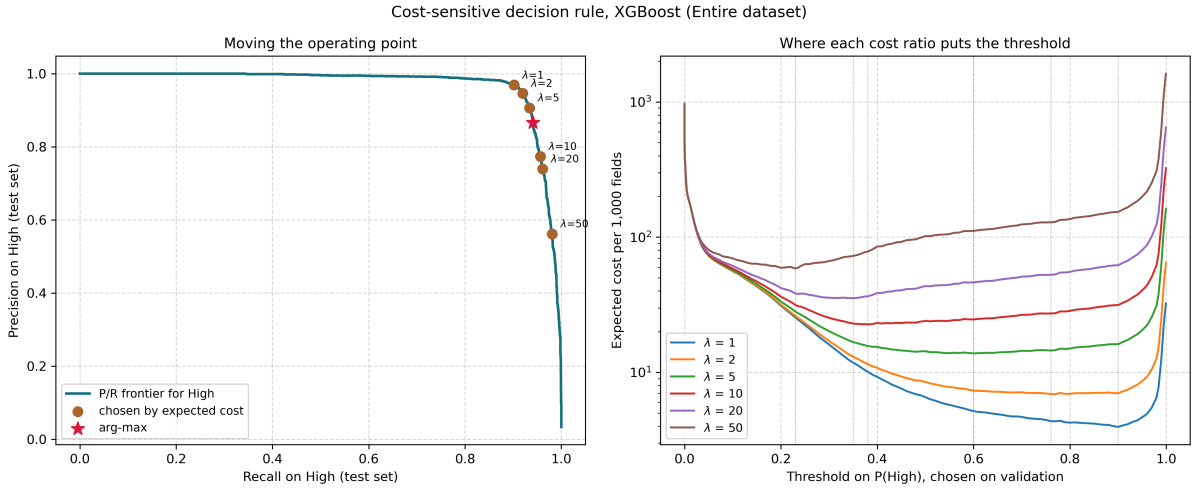

Figure 5: Left: the precision–recall frontier for `High` on the test set, with the arg-max operating point (star) and the points chosen by expected cost. Arg-max sits below the frontier's knee, which is the graphical form of the $\lambda = 1$ result. Right: expected cost per 1,000 fields against the threshold, measured on training-side validation; each curve has a distinct minimum, and the minima march leftwards as λ grows.

Two qualifications belong with the table. Macro F1 falls monotonically after $\lambda = 1$, down to 0.8843 at $\lambda = 50$: the aggregate metric penalises exactly the trade the operational criterion asks for, which is the

sharpest form of the disagreement Section 6.4 describes. And the probabilities are not calibrated. Those same inverse-frequency weights distort the scale by construction, and the Brier score of $P(\texttt{High})$ on the test set is 0.0070. The mapping from λ to t is only as trustworthy as that scale, so the table demonstrates the mechanism and the order of the trade; it is not a deployment recommendation, and Section 7 returns to it.

6.6 The Target Is a Threshold Rule

The headline result of the benchmark is not the $\approx 97\%$ but the reason it is reachable. Table 9 sweeps the depth of a single decision tree.

Table 9: Depth sweep of a single decision tree. *CV F1* is a 3-fold cross-validation on the arena's own training partition and is what selects the depth of the reference tree in Tables 6 and 7; *Test F1* is the same tree scored on the common test set, reported for description only. Both curves climb to a peak and then decay as capacity grows, which is the signature of a target of fixed complexity.

	Balanced			**Full data**		
`max_depth`	Leaves	CV F1	Test F1	Leaves	CV F1	Test F1
1	2	0.5041	0.3446	2	0.4829	0.4826
2	4	0.6729	0.5443	4	0.6388	0.6351
3	8	0.8071	0.6836	8	0.6630	0.6565
4	16	0.8481	0.7372	16	0.7327	0.7293
5	32	0.9002	0.8694	32	0.7583	0.7542
6	62	0.9558	0.9407	63	0.9028	0.9015
8	183	**0.9631**	0.9387	221	**0.9696**	0.9682
10	384	0.9597	0.9350	588	0.9687	0.9677
12	613	0.9573	0.9161	1216	0.9663	0.9666
unbounded	1521	0.9466	0.8829	9866	0.9440	0.9452

If the target were a noisy statistical relationship, macro F1 would climb with capacity and then flatten. It does not. Both curves rise to a sharp peak and then *decay*: on the full partition the unbounded tree scores 0.9452 on the test set with 9866 leaves, against 0.9682 with only 221. That is the signature of a target of fixed, finite complexity, past which the extra capacity has nothing left to fit but noise.

The table reports two curves per arena and the distinction between them matters. The cross-validated column never touches the test set and is what chooses the depth of the reference tree; the test column describes those same trees on the held-out rows. Choosing a depth from the test curve and then reporting that tree's test score would be circular, so the two are kept apart. An earlier version of this work did exactly that.

The selecting curve is unambiguous and, unlike the test curve, it agrees across the two arenas: cross-validation peaks at depth 8 in both (0.9631 balanced, 0.9696 on the full partition) and falls away on either side, down to 0.9466 and 0.9440 for an unbounded tree (Figure 6). On the test set the balanced arena appears to peak one level earlier, at 6, but by 0.0020, within what a single test set can resolve, so nothing should be read into it. The complexity this target demands is the same in both arenas. What grows with the sample is only how cleanly a tree resolves it: 183 leaves at depth 8 on 50,421 rows against 221 on 504,000.

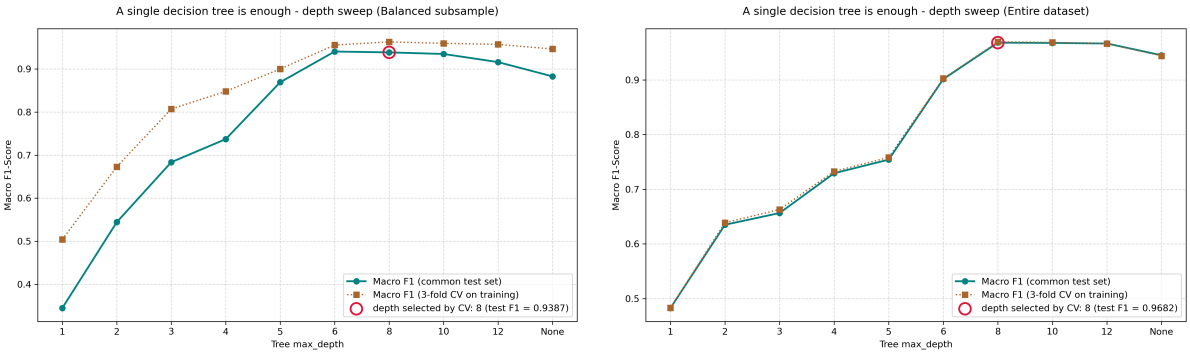

Figure 6: Depth sweep of a single decision tree: balanced subsample on the left, full training partition on the right, both scored on the common test set. Each curve climbs to a marked peak, depth 6 on the left, depth 8 on the right, and then decays as further capacity is spent on noise. A target of fixed complexity produces this shape; a noisy statistical relationship would produce a plateau.

Inspecting the splits confirms it. Converted back from standardized units, the thresholds a shallow tree selects fall on `Soil_Moisture` = 24.99, `Temperature_C` = 29.98 and `Wind_Speed_kmh` = 9.99: the round numbers 25, 30 and 10. The dataset is synthetically generated [8], and its label appears to follow a threshold rule written on exactly those values.

This reframes the entire result landscape. Axis-aligned tree models recover the rule directly, which is why they dominate; distance- and margin-based estimators must approximate a step function with a smooth boundary, which is why K-NN and both SVMs trail. The $\approx 97\%$ is an *explained outcome*, not an achievement of modelling.

6.7 Dimensionality Reduction Does Not Rescue K-NN

Section 2 identified a computational scalability wall: K-NN defers its whole cost to prediction and needs the training set resident in memory to answer a query, which is why it cannot be *tuned* on the full 504,000-row partition. Principal Component Analysis is the obvious escape route: compress the 38 encoded dimensions to a handful and the neighbour search becomes cheap. Table 10 measures whether that trade is worth making, in accuracy as well as in seconds.

Table 10: PCA + K-NN sweep, each arena at its own tuned K: 31 on the balanced subsample, 11 on the full partition. Explained variance is cumulative and given for the balanced arena; the full-data projections differ by under two percentage points. Seconds are single-run wall clock on one machine, so only their orders of magnitude carry an argument. The uncompressed row on the full partition was left empty in an earlier version of this work, on the assumption that it could not be run.

		Balanced			**Full data**		
Components	**Var.**	Macro F1	Rec. High	s	Macro F1	Rec. High	s
2	22.3 %	0.4873	0.8165	0.8	0.4329	0.0083	0.9
3	29.3 %	0.4864	0.8120	1.3	0.4363	0.0112	1.6
5	41.2 %	0.5471	0.8408	4.1	0.5452	0.1768	6.6
10	67.6 %	0.5660	0.8779	37.5	0.5770	0.2242	126.9
20	87.6 %	0.6229	0.9150	3.5	0.7243	0.3653	30.6
all (38)	100.0 %	0.6452	0.9343	3.8	0.7514	0.3755	36.7

It is not, and Figure 7 shows the shape of the failure. In the balanced arena macro F1 falls monotonically with compression, from 0.6452 uncompressed to 0.4873 at two components, and recall on `High` follows it down from 0.9343 to 0.8165. There is no knee in the curve at which a useful amount of accuracy survives a useful amount of compression: the two decline together throughout.

The cumulative explained variance shows why. Two components retain only 22.3% of the variance and three retain 29.3%; reaching 87.6% takes twenty. The encoded space simply has no low-dimensional linear structure to exploit, which is the precondition PCA needs to be worth applying.

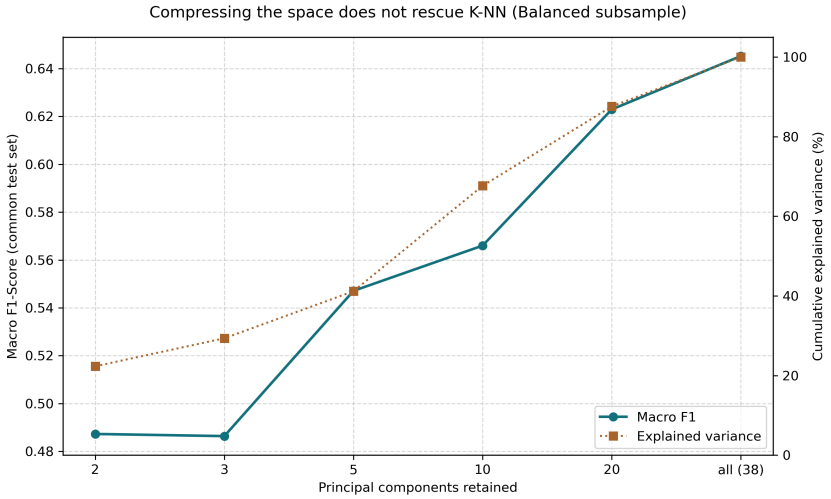

Figure 7: PCA + K-NN on the balanced arena: macro F1 against the number of retained components on the left axis, cumulative explained variance on the right. The two climb together and neither shows a knee, which is the visual form of the trade this subsection rejects.

The deeper reason connects directly to Section 6.6. PCA selects the directions of greatest variance, which have no reason to align with the axes a threshold rule is written on: rotating away from `Soil_Moisture`, `Temperature_C` and `Wind_Speed_kmh` mixes the deciding variables into combinations of themselves. Compression here is not a lossy shortcut to the same answer; it discards the structure the classifier needs.

The full-data columns add a failure of a different kind, and a severe one. Trained on the natural distribution, where `High` is 3.3% of the data and K-NN receives no class weighting, the compressed model very nearly stops predicting the rare class at all: recall on `High` is 0.0083 at two components and 0.0112 at three, meaning that of the 4,202 droughts in the test set fewer than fifty are detected. A vote among 11 neighbours drawn from a population that is 58.72% `Low` can only return `High` where the minority class is locally dense, and compression is precisely what destroys that local density. Even at twenty components recall reaches only 0.3653, and uncompressed it stops at 0.3755: the ceiling is set by the training distribution and not by the compression. This mirrors, in the opposite direction, the precision collapse of Table 6: balancing the training set makes these estimators over-predict the rare class, and not balancing it makes them ignore the class entirely.

The timing column is not even monotonic: prediction is slowest at ten components and faster at twenty, because scikit-learn uses a k-d tree up to ten dimensions and brute force above, and in between the tree pays to be traversed while already degrading towards exhaustive search. So the compute saving does not reliably arrive. Twenty components cost 30.6 seconds on the full partition against 36.7 uncompressed, which buys nothing, and the projections that are genuinely cheaper are the ones that destroy the classifier.

Figure 8 makes the same point in pictures. In the two-component projection the three classes lie almost on top of one another, and no boundary drawn in that plane could support the recall the uncompressed model reaches.

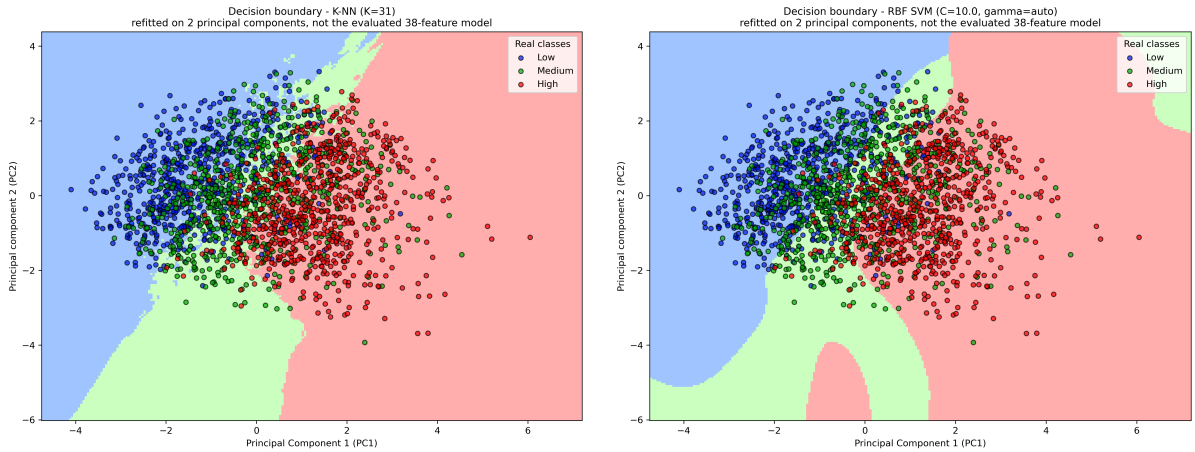

Figure 8: Decision regions of K-NN ($K = 31$, left) and of the RBF SVM (right) on the first two principal components of the balanced arena, with 2,000 training points overlaid. Both estimators are cloned from the tuned models and then *refitted on two components*: they see 2 features where every model evaluated in this section sees 38. These are illustrations of how each family carves up a projection, never the boundary on which any reported score was measured.

6.8 The Ensembles Buy Nothing

The most striking takeaway from our experiments is straightforward: on the full dataset, a single decision tree with just 221 leaves performs identically to a massive, tuned Random Forest boasting 300 trees and over 2.7 million leaves. Both models achieve a macro F1 score of 0.9682.

While agreeing to four decimal places doesn't automatically imply a true tie, analyzing their errors tells a similar story. The tree misclassifies 311 `Low` instances as `Medium`, while the forest misclassifies 328. To confirm this wasn't just noise, we ran a paired bootstrap test, resampling the test set 10,000 times. The forest scored just -0.0001 relative to the tree, with a 95% confidence interval of $[-0.0011, +0.0009]$. Because this interval cleanly contains zero, we can confidently say the models are tied in performance.

However, their operational footprints are vastly different. The Random Forest requires 494 MB of memory and takes 4.9 seconds to score 126,000 rows, whereas the single tree requires only 39 KB and scores the data almost instantly. XGBoost sits between them, taking up 1.03 MB. While it scores measurably lower on macro F1 (0.9560), it notably maintains the highest recall on the `High` class (0.941).

We were able to validate these findings externally using the competition's 270,000-row holdout set. On these unseen rows, the tree and the forest made the exact same predictions 99.84% of the time. In fact, depending on which half of the external test set was used, the two models traded the lead, further proving that they effectively learned the exact same decision boundaries.

The raw metrics translated cleanly to the external leaderboard as well. The competition ranks models using balanced accuracy (which we tracked internally as the *Bal. acc.* column in Table 7). Knowing this allowed us to predict how our models would place. Internally, XGBoost lagged in macro F1 but led in balanced accuracy (0.9671). Exactly as expected, it outperformed both the tree and the forest on the external leaderboard, scoring **0.96785** and **0.96630** across the two halves.

There is one important caveat: this equivalence between simple trees and massive ensembles only holds *on the full training partition*. When we artificially restricted the data to a balanced subsample (50,421 rows), the single tree lacked the data to grow to full depth and fell behind the ensembles. But given the full 504,000 rows, the tree perfectly recovered the underlying rule, rendering the ensembles entirely redundant.

6.9 Feature Importance and the Engineered Terms

Feature importance rankings confirmed the hypothesis we outlined in Section 5: `Soil_Moisture` is the dominant driver of the target, followed by `Crop_Growth_Stage` and `Mulching_Used`. Interestingly, the continuous variables that follow in importance (`Temperature_C`, `Wind_Speed_kmh`, `Rainfall_mm`) are exactly the features where our shallow tree from Section 6.6 placed its straightforward thresholds.

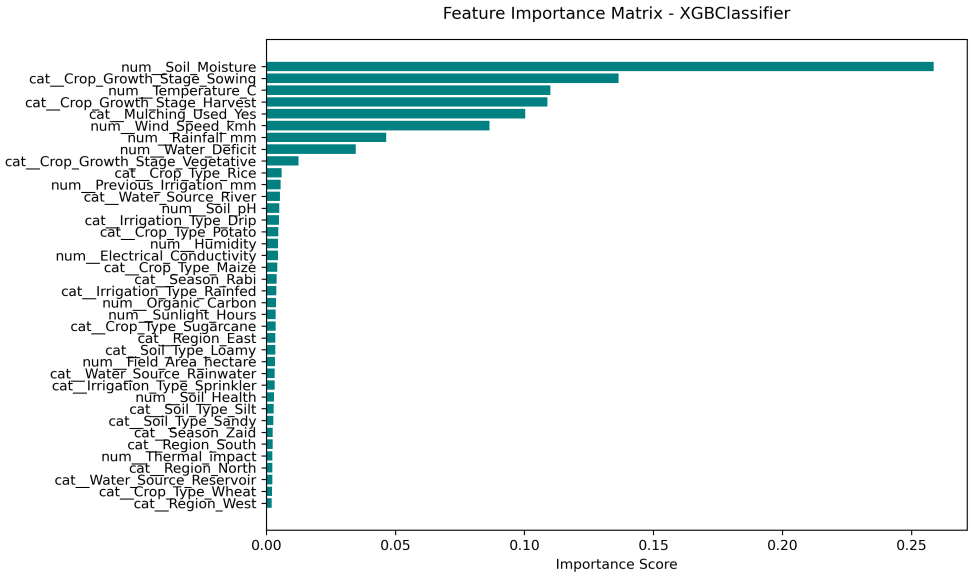

Figure 9: XGBoost importances on the full training partition. `Soil_Moisture` accounts for roughly 0.26 of the total importance—nearly double the next best feature. Notice that our engineered `Water_Deficit` feature (FE_3) ranks eighth. It is the only engineered term the model actively uses, though our ablation study (Table 11) reveals that we can safely remove it without hurting performance.

To truly test the value of our engineered features, we needed a stricter approach than simple importance rankings, which often split credit among correlated columns. In Table 11, we removed each engineered term one by one and refit the models, keeping all hyperparameters locked to isolate the impact of the features themselves.

Table 11: Feature-engineering ablation (macro F1 on the common test set). The Δ columns show the performance change relative to using the full feature set. We focus closely on the Linear SVM because, unlike tree-based models, it cannot natively construct interactions or ratios out of raw inputs.

	Lin. SVM bal.		**Lin. SVM full**		**XGB**	**Tree**
Feature set	F1	Δ	F1	Δ	full	full
All features (reference)	0.6833	—	0.7536	—	0.9560	0.9682
− all three engineered	0.6742	−0.0091	0.7510	−0.0026	0.9564	0.9682
− FE_1 Thermal Impact	0.6832	−0.0001	0.7542	+0.0006	0.9562	0.9682
− FE_2 Soil Health	0.6827	−0.0006	0.7536	+0.0000	0.9569	0.9682
− FE_3 Water Deficit	0.6748	−0.0085	0.7523	−0.0013	0.9561	0.9682
+ standalone E	0.6848	+0.0015	0.7545	+0.0009	0.9557	0.9682

The Linear SVM provides the most insight here. Because a linear model cannot independently figure out how to divide or multiply two features together, any genuine predictive power hidden in our ratios should cause a noticeable jump in the SVM's performance. Conversely, tree models can approximate these ratios by simply making repeated cuts on the base features. If we had only evaluated our feature engineering using trees, we would have incorrectly concluded that the new features were completely useless.

Using a paired bootstrap to separate signal from noise, we found that FE_1 and FE_2 offered no detectable benefit across any arena. FE_3 (Water Deficit), however, was the exception. On the balanced subsample, dropping FE_3 cost the SVM -0.0084 in macro F1, a statistically significant drop. But as we increased the training data to the full partition, this advantage shrank to just -0.0013, blending into the noise. Ultimately, while FE_3 carried some real information for the linear model in low-data regimes, the tree-based models completely ignored all three engineered features, regardless of data size.

7 Conclusions

While this pipeline successfully met our initial modeling objectives, the most valuable takeaway isn't the final F1 score—it's what the models revealed about the dataset itself.

The underlying data is governed by a simple threshold rule. This single fact explains our entire results landscape. A basic decision tree with 221 leaves tied a tuned Random Forest because the complex model simply wasn't necessary to capture the data's logic. If we had only reported the ensemble's $\approx 97\%$ F1 score without this context, it would have falsely implied that the dataset required a highly sophisticated, computationally expensive solution.

Aggregate metrics can mask operational failures. When evaluated on the full data, the Linear SVM achieved a higher macro F1 score than K-NN. However, looking under the hood revealed that the SVM missed nearly 63% of actual droughts, misclassifying them into a category that would fail to trigger an operational response. By tracking per-class recall alongside our aggregate F1 scores, we caught a flaw that would have been disastrous in a real-world deployment.

External validation proved our internal evaluations were accurate. The Kaggle holdout set confirmed our most surprising internal findings. The external test data showed that our single tree and Random Forest were functionally tied, making the exact same predictions in over 99% of cases. Furthermore, as our internal recall metrics predicted, XGBoost outperformed both on the competition's balanced accuracy metric.

Class balancing is a trade-off, not a free lunch. When we trained models on a balanced 1:1:1 subsample, they successfully caught most droughts (recall above 0.93) but generated a flood of false alarms, causing precision to collapse. Training on the full, imbalanced dataset flipped this dynamic. Confining our balancing efforts strictly to the training phase allowed us to honestly measure this exact trade-off against the true real-world distribution.

Engineered features must justify their complexity. Our ablation study showed that two of our three engineered features provided no measurable value. The third feature (Water Deficit) only helped a linear model, and even then, only when data was limited. The broader lesson here is that an engineered feature only earns its place if the model cannot organically reconstruct that relationship itself, and only if its contribution stands out above the statistical noise.

Limitations and Future Work

We should acknowledge a few constraints in our methodology. Several of our best hyperparameters hit the very edges of our search grids, suggesting that the true mathematical optima might lie further out; expanding the search grid for the SVM penalty and the number of trees would be a logical next step. Additionally, our decision-boundary visualizations rely on 2D principal components, which only illustrate the general geometry of the models rather than the exact high-dimensional boundaries.

More importantly, these findings are bounded by the nature of the data. Because this dataset is synthetic, the underlying "threshold rule" behavior is an artifact of how the data was generated. We cannot guarantee that ensembles would be equally redundant on real-world agricultural telemetry without re-testing.

Finally, our cost-benefit analysis in Section 6.5 relied on uncalibrated probability outputs. Moving forward, applying a probability calibrator to the validation slice would likely refine (and perhaps shift) our suggested thresholds.

References

[1] J. V. Stafford, "Implementing precision agriculture in the 21st century," *Journal of Agricultural Engineering Research*, vol. 76, no. 3, pp. 267–275, 2000.

[2] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[3] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[4] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pp. 785–794, 2016.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[8] Kaggle, "Playground series – season 6, episode 4: Predicting irrigation need." Kaggle Competition, 2026. Synthetically generated from the *Irrigation Water Requirement Prediction* dataset.

[9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.